FunnelStream defense: 100 likely questions and answers

This list is for rehearsal. Every answer is based on the repository and on `docs/FunnelStream-Project-Documentation.md`. Where the repository has no written reason for a choice, the answer says so and gives the honest reasoning instead of inventing a history.

Three rules for every answer

1. Say that the data is simulated whenever a number is quoted.
2. Separate what is implemented, what is a design choice, and what is future work.
3. If you do not know, say what the repository shows and offer to look at it. Do not guess a figure.

Sections: A. Business and scope (1-10) | B. Architecture and technology choices (11-25) | C. Data sources and the simulator (26-33) | D. Kafka and streaming (34-42) | E. Bronze, Silver and data quality (43-55) | F. Gold, Databricks and attribution (56-66) | G. Grafana, alerts and inventory (67-78) | H. Power BI and metrics (79-88) | I. Testing, performance and reliability (89-95) | J. Security, limits, future work and process (96-100)

A. Business and scope

1. What problem does FunnelStream solve? An online store spends money on advertising, ships mostly cash on delivery, and promises fast delivery from dark stores. It cannot easily answer three questions in one place: which campaign produces delivered orders, where shoppers drop out of the funnel, and what needs action right now. FunnelStream joins the events behind all three into one platform with two views: a live one for operations and a next-morning one for marketing.

2. Who are the users? There are two. The operations user watches live traffic, stock in eight dark stores and the 20-minute express promise in Grafana. The marketing user reads yesterday's campaign, funnel and sales results in Power BI.

3. What is the one thing only a joined platform can show? Advertising spend that was sent to products that were out of stock. Operations sees it live as demand that arrived at an empty shelf. Marketing sees it the next morning as estimated exposed spend.

4. Is this real business data? No. Customers, orders, ad numbers and stock are generated by a seeded simulator. The numbers demonstrate the pipeline and the method. They are not real business performance, and the dashboards say so.

5. Why simulate instead of using a public dataset? We needed streaming events, orders with a lifecycle, ad data, and stock events that all agree with each other, and we needed to inject known faults and count them. A public dataset does not give a consistent stream with stock, express delivery and campaign links. The seed also makes every demo and replay test repeatable.

6. How realistic is the simulated data? It models an Egyptian store: Egyptian names and phone numbers, cities weighted by population with real districts, EGP, cash on delivery as the largest payment method, Fawry,

Vodafone Cash and InstaPay, the carriers Bosta, Aramex and J&T, and Ramadan and Eid peaks. We cannot prove it is fully realistic. The assumptions are documented and the distributions are checked against them.

7. What business decisions can the system support? Moving budget between campaigns using ROAS on delivered orders, fixing the funnel step and channel with the highest abandonment, restocking products that ran out, chasing express orders about to miss their promise, and pausing ads for products that are out of stock. All of these are shown on simulated data.

8. What is the scope, and what did you deliberately leave out? In scope: ingestion, streaming processing, quality checks, a lakehouse model, live monitoring with alerts, a seven-page BI report and an LLM question assistant. Left out on purpose: real customers, a cloud deployment of Kafka and Spark, recommendations (the `rec_product` table is an empty slot), incrementality testing and contribution margin.

9. Why two dashboard tools, Grafana and Power BI? They serve different users and speeds. Grafana reads small Postgres tables refreshed within a minute and has alert rules, so it fits operations. Power BI reads the reconciled Gold tables and the daily marts, so it fits marketing, who need trusted numbers for yesterday. Grafana answers "is it healthy and what needs action now"; Power BI answers "what happened and what should we decide".

10. What is the headline result? A streaming pipeline that survives a crash without duplicates, bad data that is quarantined with a reason and counted against the injected noise, and two audiences served from the same events with one bridge between them. The measured proof points are 66 data quality checks with no failure, 1,169 host tests passing, and `NO DUPLICATES` in two kill-and-restart runs.

B. Architecture and technology choices

11. Walk me through the architecture in one minute. A simulator and a mock ad API produce events. Kafka carries four topics. A Spark Structured Streaming job in Docker writes Bronze (raw) and Silver (validated and deduplicated) Delta tables, with a quarantine table for rejected rows. From Silver, a live lane writes small Postgres tables for Grafana and eight alert rules. A batch lane publishes Silver to Databricks, builds the Gold star schema with 66 checks, adds daily marts in Postgres and feeds Power BI.

12. Why a medallion (Bronze, Silver, Gold) design? Each layer has one job. Bronze keeps every raw message forever, so nothing is lost and layers can be rebuilt. Silver is the one trusted copy: typed, validated and deduplicated. Gold is business ready, with facts, dimensions and aggregates. Separating them lets us fix a rule in Silver or Gold and rebuild without asking the source again.

13. Why two lanes instead of one pipeline? Decisions need different speeds and different levels of certainty. Operations needs a number within a minute even if a late event may still change it. Marketing needs a number that reconciles and does not move. The live lane trades certainty for speed and leaves late events out. The batch lane includes them and runs 66 checks.

14. Why Kafka? It separates producers from processing, keeps messages while the Spark job is down, and lets us replay after a crash. Keys keep per-session and per-order ordering. Kafka also runs locally in one container, which suited a laptop deployment. We did not benchmark other brokers, so the honest claim is that Kafka fits the needs, not that it beat alternatives.

15. Why Spark Structured Streaming and Delta Lake? They give checkpoints, partitioned tables and a Delta `MERGE`, which together make writes idempotent, and the same code can replay Bronze to rebuild Silver. We did

not evaluate Flink or ksqlDB in this project, so we make no comparative claim.

16. Why does Spark run in Docker? On this Windows host Spark needs winutils and a Linux-like environment, so the Spark jobs and the Spark tests run in a container. Java and Spark live inside the Docker image and are not installed on the host.

17. Why Databricks for Gold? It gives a SQL warehouse and Unity Catalog for the star schema, and it keeps the cloud part of the design visible: the local lake holds Bronze and Silver, and Databricks holds the Silver mirror and Gold. It is a Free Edition workspace, so it has limits, and our benchmark is one setup, not a capacity claim.

18. Why Postgres for the live tables? Grafana needs small, fast, indexed tables that it can query every few seconds. The live sink recomputes only the minutes and store and product pairs a batch touched and writes them to Postgres. Postgres also holds the daily marts that Power BI reads. It is not the analytical store.

19. Why Grafana for monitoring? It provisions dashboards and alert rules as code, supports a read-only data source, and handles gaps and thresholds well. Six dashboards and eight alert rules are generated by Python builders, and a test fails if a file drifts from its builder.

20. Why Power BI for the business layer? It is the common business tool, it supports a semantic model with measures and relationships, and the project format (`.pbip`) keeps the report in text files that a generator can produce and a test can check. We used Import mode with read-only credentials.

21. What is the role of Airflow? One DAG, `ads_yesterday_report` , runs at 03:00 UTC, pulls the ad API once for the finished UTC day, stores a snapshot and writes a JSON report. It retries on HTTP 503 and returns the stored report on a rerun. It is an optional Docker profile, and the same steps also run as a Python script.

22. Why does the Airflow data go straight to Silver and not Bronze? Bronze holds raw Kafka messages with partition, offset and ingest time. The ad report does not come from Kafka. It is one HTTP pull a day, validated when pulled and stored in `silver_campaign_snapshots` . Every pull is kept as its own snapshot, keyed by campaign, report date and pull time, and bad rows go to quarantine. The trade-off is that the raw API response is not kept. A `bronze_ad_pulls` table would close that gap, and that is the honest next step.

23. Is the architecture the same as it would be in production? No. Everything left of Databricks runs on one Windows laptop in Docker. In production we would use managed Kafka, object storage for the lake, a cluster for Spark, a secrets manager, CI and monitoring of the platform itself. These are proposals, not implemented features.

24. What are the design principles? Medallion layers, at-least-once delivery with idempotent writes, a single writer per table, two lanes, contracts at the door (a schema validator), declared metric definitions in one dictionary, and fallbacks such as a Postgres store behind the same interface as Databricks.

25. What does "single writer per table" mean and why does it matter? Every Gold table is written only by the Databricks Gold build, and the local Spark stream writes only Bronze, Silver and the live tables. One writer per table removes write conflicts and makes it clear where a wrong number came from.

C. Data sources and the simulator

26. What events does the simulator produce? App and web events with funnel drop-off (`app_open` , `page_view` , `product_view` , `product_click` , `product_search` , `add_to_cart` , `remove_from_cart` ,

`checkout_started`, `purchase`), orders and deliveries with a full lifecycle, stock snapshots and movements for eight dark stores, and campaign numbers from a mock Meta and Google Ads API. The catalog has about 200 products in four categories and 5,000 customers.

27. What is in an event? Every event uses one envelope: `event_id`, `event_type`, `event_version`, `event_time` and a `payload` object. Anonymous events have no `customer_id` and are keyed by `session_id`.

28. Why inject noise on purpose? A clean stream proves nothing about data quality. The simulator injects duplicates (1%), malformed JSON (0.3%), missing fields, unknown types, negative prices, future timestamps, late events (2% plus 0.3% very late), dirty text, bots, out-of-order order status and bad ad rows. It writes the counts to `noise_manifest.json`, so we can check that the pipeline caught what was injected.

29. How do you know the pipeline caught all the noise? After Silver, the quarantine count is compared with the noise manifest and must match within a tolerance. On the 13 October scratch run, 308 rows were expected and 308 were observed.

30. What does "seeded" mean? The same seed gives the same customers, products and events. That makes a demo repeatable and lets replay tests compare results. The demo script picks a new seed each run so ids never repeat.

31. Is the ad API real? No. It is a mock of a Meta and Google Ads report API. Its numbers are consistent with the `utm_campaign` values in the generated traffic. It can also fail on purpose (HTTP 503, duplicate pulls, `clicks > impressions`, negative spend) to test the DAG and the validator.

32. Why eight dark stores? They map the quick-commerce case to real Egyptian districts: Nasr City, Maadi and New Cairo in Cairo, Dokki in Giza, Montaza and Raml in Alexandria, one in Mansoura and one in Tanta. Each district has one primary store. Assiut and "other" have no store and are excluded from out-of-stock metrics.

33. Does the simulator model stock realistically? It has a `StockLedger` that never lets stock go below zero, emits an opening snapshot for every store and product pair, a baseline snapshot every five minutes and a `sold` movement for each order line. Demand events are not stopped by empty stock, because that lost demand is what we measure. Known limit: stock is reserved when an order is placed and is not put back on cancellation, return or failed delivery.

D. Kafka and streaming

34. What topics do you have and why those four? `app.events` (6 partitions, key `session_id`), `orders` (3, key `order_id`), `delivery.events` (3, key `order_id`) and `inventory.events` (3, key `store_id`). They split by entity and by ordering need, so each key keeps its events in order within one partition. `app.events` has 6 partitions because it is by far the busiest stream, and the other three topics have 3 because they carry a small fraction of that traffic. The repository records the numbers but not a written reason, so say that they were chosen in proportion to traffic and sized for one laptop (the Spark job runs with `local[4]`, four parallel tasks), and that we did not measure other values. In production the count would come from peak throughput and the number of consumer tasks, with headroom, because a partition count can be raised later but not lowered.

35. Why those keys? All events of a session, and all events of an order, go to one partition in sequence. That keeps per-session and per-order ordering, which the order lifecycle logic relies on.

36. What delivery guarantee do you have? At-least-once delivery with idempotent writes. Kafka producers use `acks=all` and idempotence, Silver deduplicates on `event_id`, and Delta `MERGE` on the key means a replayed message inserts nothing. The result is exactly-once in effect, without a broker-level exactly-once setup.

37. What happens if the Spark job crashes? Checkpoints on disk hold the committed Kafka offsets. When the job restarts with the same command and the same checkpoint folder, it resumes from those offsets and reads what Kafka kept while it was down.

38. Did you test that? Yes. In the failure demo we killed the Silver job with `docker kill`, watched the "Silver job silent" alert fire 2 min 12 s and 2 min 54 s after the kill in two runs, restarted with the same checkpoints, and the duplicate check reported `NO DUPLICATES` both times. The check covers Silver only, and it does not prove that no event was lost.

39. What is the micro-batch trigger? Ten seconds. On this laptop Silver batches can be slower than the trigger and Spark logs a "batch is falling behind" warning. That is normal here, and we keep the Gold sync off during a demo.

40. How is event time handled? Events carry `event_time` in UTC. An event is late when Kafka time minus event time exceeds 10 minutes. Late events stay in Silver and Gold, are flagged `is_late`, are left out of the live metrics, and are corrected by the batch. Business days are taken in `Africa/Cairo` time.

41. How do you handle out-of-order order statuses? Arrival order is not trusted. The current status of an order is the status with the highest rank: `order_created` 1, `payment_authorised` 2, `order_packed` 3, `order_shipped` 4, `out_for_delivery` and `delivery_failed` 5, `delivered` 6, `order_completed` and `order_returned` 7. `order_cancelled` has rank 2 and is terminal, and a cancel after shipment is rejected to quarantine.

42. Why not read directly from Kafka into Postgres without Bronze and Silver? We would lose the raw copy, deduplication, the quarantine, replay and the Silver tables that feed Gold. Bronze and Silver are what make crash recovery, rebuilds and the quality proof possible.

E. Bronze, Silver and data quality

43. What is in Bronze? The raw message value plus the Kafka partition, offset, timestamp and the ingest time. It is append-only and nothing is dropped or changed. It is the source for any rebuild.

44. What does Silver do? It validates against a versioned schema, types the fields, normalises (UTC, city and device text, UTM values), deduplicates on the key and writes with a Delta `MERGE`. Every Silver table has a key, so replaying a batch changes nothing. A batch is deduplicated before the `MERGE`, so two rows with the same key never meet in one merge.

45. What goes to quarantine? Rows that fail validation, with an error code and the raw payload: `BAD_JSON`, `MISSING_FIELD`, `UNKNOWN_EVENT_TYPE`, `BAD_VERSION`, `NEGATIVE_PRICE`, `FUTURE_TIMESTAMP`, `UNKNOWN_PRODUCT`, and for ads `NEGATIVE_SPEND` and `CLICKS_GT_IMPRESSIONS`. A quarantined row never reaches the numbers.

46. Why quarantine instead of dropping or fixing bad rows? Dropping hides a producer problem. Guessing a fix invents data. Quarantine keeps the evidence, gives a reason that points at the producer to fix, and lets us count rejected rows against the injected noise. The quarantine share also drives an alert at 2%.

47. Can Silver be rebuilt? Yes, from Bronze at any time, for everything that comes through Kafka. The ad snapshots are the one exception, because they do not pass through Bronze. We replayed Bronze in the smoke test and 10 Silver tables were unchanged.

48. How do you remove duplicates? The simulator duplicates 1% of events with the same `event_id`. Silver deduplicates on `event_id` inside the batch and the `MERGE` ignores a key that already exists. `scripts/check_duplicates.py` reads every keyed Silver table and reports duplicates.

49. What are the Silver tables? `silver_events`, `silver_orders`, `silver_order_status`, `silver_order_items`, `silver_stock_movements`, `silver_stock_snapshots`, `silver_express_orders`, `silver_customers`, `silver_products` and `silver_quarantine`, plus `silver_campaign_snapshots` from the ad API.

50. What are bot sessions and what do you do with them? About 0.1% of sessions are bots with hundreds of events in seconds. They get an `is_bot` flag and are left out of the funnel, attribution and live metrics. A known limit is that the out-of-stock demand path does not remove bot sessions, while Gold does.

51. How do you validate the data contracts? A validator in `contracts/` checks every event against a versioned schema for its type. A bad event goes to quarantine with an error code. The validator and the schemas are covered by tests.

52. What are the 66 data quality checks? 66 results on every Gold build: 46 fail-type checks that stop the batch when they find a violation, and 20 report-type checks that only print a number (rates and row counts). Groups: duplicate keys, Silver hygiene (no null `event_id`, nothing in the future, no unknown product, `line_total = quantity x unit_price`), hierarchy and history (one current row per dimension key, no overlapping validity, district belongs to its city), reconciliation (order totals equal their lines, Gold counts match Silver), the manifest check (quarantine within 2% of the manifest) and reported rates.

53. What does the manifest check prove? That the pipeline's quarantine count equals the faults the simulator says it injected, within 2%. It is evidence that the validator is neither missing faults nor rejecting good rows. It works because the simulator is our own, so we know the true count.

54. What happens to late events? They are kept. They are flagged, excluded from live metrics so the live lane does not move after the fact, and included in the batch so Gold and Power BI are correct.

55. How is personal data handled? Names and phone numbers exist only in Bronze and Silver. Publishing to Databricks uses an allow-list of columns, so they never reach the Silver mirror, Gold or Power BI. Gold and the report hold `customer_id` only. All data is synthetic.

F. Gold, Databricks and attribution

56. What is in Gold? Twelve tables: `dim_customer` and `dim_product` (slowly changing, type 2), `dim_campaign`, `dim_date`, `fact_orders`, `fact_order_lines`, `fact_attribution`, `agg_orders_daily`, `agg_funnel_daily`, `agg_campaign_daily`, `agg_city_product_daily` and `customer_360`. A thirteenth slot, `rec_product`, is empty by design.

57. What is SCD2 and where do you use it? Slowly changing dimension type 2 keeps history by writing a new row for each change, with `valid_from`, `valid_to` and `is_current`. `dim_customer` tracks city, district and primary device. `dim_product` tracks category, price and active flag. The simulator changes prices and

categories on purpose so the history exists. Checks ensure exactly one current row per key and no overlapping validity.

58. How do facts find the right dimension row? Dimension keys are resolved when the fact is built, from the dimension version valid at event time. A row without a match goes to quarantine and does not get a guessed key. Key -1 is the unknown member of the dimensions.

59. How do you publish Silver to Databricks? Databricks cannot reach the laptop's Kafka, so Silver crosses as files. `scripts/publish_silver.py` exports Silver with an allow-list of columns, uploads Parquet to a Unity Catalog volume and `MERGE`s it into the Silver mirror. It is insert-only and idempotent. An optional `--reset` truncates the owned mirror tables before a fresh publish.

60. How does attribution work? There are two models. `last_touch` gives an order to the campaign of its own session click first, otherwise to the customer's last click in the previous seven days. `first_touch` gives it to the first click. An order with no click is direct, with a NULL campaign.

61. Why is the headline ROAS on delivered orders? Many orders are cash on delivery and are often cancelled or fail delivery, so ROAS on placed orders overstates the return. The headline is `revenue_delivered / spend`, and placed ROAS is secondary. Delivery lags spend by about two to four days, so recent days read low and are labelled provisional.

62. Does attribution prove the campaign caused the order? No. Last touch favours bottom-funnel channels such as search and retargeting, and neither model proves incrementality. The report states both models. Incrementality tests are listed as future work.

63. What is the difference between CPA and cost per delivered order? CPA is spend divided by placed orders. Cost per delivered order is spend divided by delivered orders only. They differ by definition. For example, on the campaigns page ramadan_offer_1 shows a CPA of 171 and a cost per delivered order of 377.

64. Why are ad numbers on a different day from orders? The ad platform reports on its UTC day, and orders belong to the Cairo day. They differ by at most three hours, and the mismatch is accepted and stated on the pages where it matters.

65. What is the Postgres fallback for Gold? A `GoldStore` interface has a Databricks backend and a Postgres backend, so the Gold tables can load into Postgres if Databricks is unavailable. The Airflow steps also run as a Python script.

66. What did the Gold build produce in the last run? On the scratch catalog on 13 October the build finished `BATCH OK` in 175.6 seconds with 66 checks and 0 failures, and the quarantine count matched the manifest (308 expected, 308 observed).

G. Grafana, alerts and inventory

67. How many dashboards and alerts do you have? Six dashboards: live pipeline, operations live inventory, express SLA, live stock monitor, inventory insights daily and marketing D-1. Eight alert rules: late-event rate, quarantine share, Silver job silent, no events, demand on out-of-stock products, express order missed its promise, express on-time rate below target and many products out of stock in one store.

68. How are dashboards and alerts managed? As code. Python builders in `monitoring/` generate the dashboard JSON and the alert rules, Grafana provisions them from files, and a test fails if a file drifts from its

builder. After changing a dashboard, the Grafana container must be restarted.

69. What does the "Silver job silent" alert do? It fires when the Silver heartbeat is older than two minutes, and no data also fires. In the failure demo it fired 2 min 12 s and 2 min 54 s after the kill, so the promise we make is "within 3 minutes".

70. How does the system define out of stock? A product is out of stock when `available <= 0`, and low when `0 < available < reorder point`. Stock is simulated, and the live tables show the current state and a per-minute snapshot per store kept for 24 hours.

71. How is the express promise measured? Express orders carry their own `promised_minutes` (20 in production). On time means delivered within the promise. Breached means delivered later, or still undelivered beyond the promise plus one minute of grace. At risk means undelivered with at least 75% of the promise used. The target is 95%.

72. Why does the Express SLA screenshot show 6 breaches? It is from a rehearsal where the promise was shortened to 2 minutes and some express orders were forced late on purpose, so that a breach and its alert can be shown live. It is not a result of the system.

73. Why is events per minute 0 in the pipeline screenshot? The screenshot was captured at the end of the 13 October rehearsal, after the generator had stopped. The tile shows the last full minute, and it is the case the "no events" alert is built to detect.

74. Why are the Gold update and publish tiles orange, showing 1 day? In that capture the last Gold update and publish were a day old, because the batch lane runs on demand and not continuously.

75. What is the "hero scenario"? A demo mode (`--scenario hero-stockout`) that makes three hero products at the busiest store start with 40 units, boosts their views and add-to-carts, and restocks them about 90 seconds after the stock-out. The same seed gives the same demo.

76. What did you measure in the demo timeline? On 12 October the first hero product reached zero about 4 minutes after the demo command (Galabeya 61 at Nasr City, 21:16:14 Cairo time). The out-of-stock demand alert fired 1 minute after, and the restock came about 2 minutes after the stock-out. An express rehearsal had 33 express orders, 31 on time and 2 breached.

77. How does the live sink work? After each Silver batch it recomputes only the minutes and the store and product pairs the batch touched and writes them to Postgres. Grafana reads them with the read-only role `grafana_ro`.

78. How do you avoid false alarms from empty data? Rate alerts need a minimum count (at least 20 events or rows, at least 5 express orders). Missing data fires only the alerts built for silence. Dashboards show gaps instead of zero for rates and stock levels, and a neutral dash for no data.

H. Power BI and metrics

79. How is the Power BI model built? A star schema in Import mode: four dimensions (`Date`, `Campaign`, `Store`, `Product`) and the facts around them. It has 19 tables, 27 relationships and 103 measures. There is no many-to-many relationship and no bidirectional filter. After the 13 October redesign it has 106 visuals across seven pages.

80. What are the seven pages? Executive, Campaigns, Marketing and stock, Operations, Funnel, Products and cities, and Customers. Each page answers one question and has at most six cards and three slicers.

81. Where does Power BI get its data? Eleven Databricks Gold tables and seven Postgres marts, with read-only credentials. Credentials are never stored in the file, so Power BI Desktop asks for the Databricks token and the Postgres password when opening.

82. Why Import mode and not DirectQuery? The repository records the choice (Import mode) but not a written reason, so answer with the reasoning: Import mode keeps the report fast and works without a live connection during the defense, and the data is refreshed after a Gold batch and the two Postgres builders. DirectQuery would depend on the Databricks warehouse being awake and would be slower.

83. Why did the report change design? We redesigned it around one identity: a navy rail with the page navigator and slicers, a title banner that states the question of the page, the Accessible Tidal colour palette, consistent spacing, ranked bars instead of pie charts, and readable tables with explicit column widths. The report is generated by `scripts/make_pbip.py`, and tests check that every visual's fields exist in the model.

84. What is "exposed spend" exactly? `exposed_spend = spend x oos_click_share`, where `oos_click_share` is out-of-stock landing clicks divided by served landing clicks. The ad API only gives spend per campaign per day, so we assume spend is spread evenly over served landing clicks. It is an estimate, not a measured loss, and not "wasted", because a shopper may buy a substitute.

85. What is "revenue at risk"? Add-to-carts on out-of-stock products multiplied by price. On the Marketing and stock page it is 71,653 on simulated data, and all three at-risk products are in one store, Nasr City.

86. Why is there a one-day peak in the revenue chart? The simulated data has a volume spike on 9 October, which holds about 55% of all revenue. The Executive page states it in its subtitle so nobody reads it as a real trend.

87. How are the abandonment rates calculated? Cart abandon rate is `1 - orders_placed / cart_sessions`, and checkout abandon rate is `1 - orders_placed / checkout_sessions`. They are blank for an empty slice and clamped between 0 and 1, never 100%. On the Funnel page they are 69.1% and 30.7%, and session conversion is 10.4%.

88. Why does the report say "settling"? Delivered revenue of recent days can still grow as orders are delivered, so `Maturity Label` marks days as "Settling" or "Settled". This is why recent ROAS delivered reads low.

I. Testing, performance and reliability

89. How is the project tested? Four levels: 1,169 host tests passed with 0 failures (13 October), 217 Spark tests in 22 files passed in Docker (12 October), a 15-stage smoke test, and a benchmark and failure demo. Tests cover contracts, the validator, the simulator, Silver logic, Gold SQL, the 66 check definitions, Grafana JSON and alert rules, the Power BI generator and the LLM guards.

90. Why are 218 tests skipped? They are the Spark tests, which need the Docker environment and run there one file at a time. Some other tests also skip when Postgres is not running.

91. What is the benchmark result? 1,000,000 simulated events (1,022,333 Kafka messages) went through the whole pipeline in 654.73 seconds on one laptop with 12 CPUs and Docker limited to about 8 GB, using a

Databricks Free Edition SQL warehouse. That is about 1,560 messages per second end to end. It measures one setup and is not a capacity claim.

92. Will it scale? We did not test beyond the benchmark. The design points that help scaling are partitioned topics, partitioned Delta tables, idempotent writes and a separate batch lane. Real scaling would need a cluster, managed Kafka and object storage, which we have not built or measured.

93. What did the smoke test show? Its first complete run on 13 October passed 10 stages and failed three, and none was a pipeline defect: a data invariant that wrongly required a live table during a replay, a cast in the test's stand-in for a Grafana time macro, and old rows in the scratch catalog. All three were fixed. The Gold stages then passed in a manual rerun. One uninterrupted green run including the Spark stage is still open.

94. What was the hardest problem? Making every step safe to repeat. The pipeline had to survive a crash and a replay without duplicates, handle late and out-of-order events, and still give numbers that reconcile. The proof is the duplicate check after a crash and the Bronze replay that left all 10 Silver tables unchanged.

95. What happens if Databricks is down? The live lane and the local lake keep working, because they do not depend on it. The Gold tables can load into Postgres through the second store backend. Power BI would show the last imported data.

J. Security, limits, future work and process

96. How do you handle secrets and access? `.env` is never committed, and `.env.example` lists names only. Kafka, Postgres, Grafana and Airflow ports are bound to `127.0.0.1`. Grafana and Power BI use read-only credentials, and the Databricks write token is used only by the load jobs. The smoke test refuses to touch the real catalog and works in a scratch catalog.

97. What are the main limitations? All data is simulated. The out-of-stock flag is the stock when the minute was written, so a replay or a late event could flag it differently. Stock is not put back on cancel, return or failed delivery. Exposed spend is an estimate. Ad numbers use the UTC day and orders the Cairo day. Gold has City but no Store dimension. Express KPIs exist only in the Postgres tables. The benchmark is one setup on Databricks Free Edition.

98. What would you do next? Compute stock intervals from Silver by event time so the out-of-stock flag is replay-safe, put stock back on cancel and return, publish the stock tables to Databricks with a Gold table for out-of-stock demand, add delivery zones and a Store dimension in Gold, fill the `rec_product` recommendations table, add margin after cash-on-delivery and cancellation cost, run incrementality tests, overlap simulator rounds so live traffic never goes quiet, and add Great Expectations or dbt on top of the SQL checks.

99. What does the LLM assistant do and how is it kept safe? It lets a user ask a question in natural language, for example which campaign had the highest CTR. The model writes one `SELECT` at a time that runs against read-only DuckDB views. File access is switched off, queries have a 20-second limit and a 200-row cap, results are wrapped in `<query_result>` tags so data cannot pass as instructions, the model never receives names or phone numbers, and the API key is never logged. The guards are tested.

100. How big is the project, and how was it built? About 30,000 lines of Python (about 17,800 of them tests) in more than 100 commits, with 88 test files. Work was done on the branch `funnelstream-build` with

documentation, a metric dictionary, runbooks, a handoff file and a pre-defense checklist in the repository. If asked who did what, answer for your own part and be specific.